

Images may contain vulnerabilities: If you're pulling public images from Docker Hub, you may be using outdated libraries with known security issues.

Misconfigured containers can be risky: Running containers with root privileges or open ports can expose your system.

Here are some friendly tips to keep your containers safe.

Use trusted base images: Start with official or verified images whenever possible.

Scan images for vulnerabilities: Tools like Docker Scout, Trivy, and Snyk can analyse your image and flag known risks.

Avoid running as root: Use Docker's rootless mode or configure your container to run as a non-root user.

Keep Docker and dependencies updated: Just like your phone or computer, Docker gets security updates too—don't skip them!

Use minimal images: Smaller images like Alpine Linux reduce the surface area for attacks.

Exploring alternatives: Podman and containerd

While Docker is incredibly popular, there are other tools gaining traction, especially in enterprise and cloud-native environments.

Podman is a drop-in replacement for Docker (you can even use the same commands). It runs containers without a central daemon (which means fewer risks). Designed with rootless containers in mind from the start, it works well on Linux and now supports Windows and macOS too.

Containerd is a lightweight container runtime originally built by Docker but is now a core part of Kubernetes. It doesn't have a full developer-facing CLI like Docker, but it's used under the hood in many systems. It's great for production systems where performance and control matter more than convenience. Here's how to choose between Docker, Podman and containerd.

- Use Docker when you're getting started, developing apps, or building locally.
- Use Podman if you need rootless security or a Docker replacement in restricted environments.
- Use containerd when building scalable, production-grade platforms (especially in Kubernetes clusters).

There's no "one size fits all," but the good news is: all these tools speak the same container language. Learning Docker gives you a strong foundation for working with any of them.

Scaling up with Kubernetes: Container orchestration demystified

So far, we've explored how containers help you package and run apps more efficiently—and how Docker makes it easy to work with them. But what happens when you go from running one container to running hundreds or even thousands? That's where Kubernetes comes in.

Kubernetes (often shortened to K8s) is a powerful system for orchestrating containers. In simple terms, it helps you manage, scale, and automate the deployment of containerised applications. Think of it like a traffic controller, ensuring all your containers are running smoothly, restarting them if they crash, and balancing the load across your infrastructure. Originally developed by Google, Kubernetes is now maintained by the Cloud Native Computing Foundation (CNCF) and is used by companies of all sizes.

Core concepts (without the jargon)

Let's break down some of the essential building blocks of Kubernetes.

Pod: The smallest unit in Kubernetes. A pod usually runs a single container, but it can run multiple if needed (they share resources).

Deployment: Tells Kubernetes how many copies (or 'replicas') of your app to run and how to update them safely.

Service: A stable way to access your pods. Even if the pods change, the service keeps the app reachable.

Ingress: Manages external access to your services, like routing traffic from a browser to your backend.

These concepts may sound a bit abstract at first, but they give Kubernetes the flexibility to manage apps of any size.

A simple real-world example

Let's say you're deploying a basic website with a backend and a database. In Kubernetes, you could:

- Create a deployment for your web server.
- Create another deployment for your database.
- Expose the web server using a service.
- Route external traffic using an ingress.

All of this is described in simple YAML configuration files. For example, a tiny snippet to deploy a container may look like:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: web
```